Assignment - 8

FP-Growth Algorithm for Association Rule Mining

Objective

The objective of this lab is to implement the FP-Growth algorithm from scratch and understand the complete frequent-pattern-mining process:

Transactions → Item Frequency Ordering → FP-Tree Construction → Conditional Pattern Bases/Trees → Frequent Itemsets → Association Rules

Students will implement FP-Growth using Python and NumPy without using a ready-made frequent-pattern or association-rule-mining function. The work should demonstrate support, confidence, frequency ordering, node-link connections, conditional pattern bases, conditional FP-trees, and rule generation.

Software Requirements

- Python 3.x
- NumPy
- Matplotlib
- scikit-learn (only for loading the built-in Iris dataset)
- Jupyter Notebook / Google Colab / Python IDE

Instructions

- Complete all tasks in a single Jupyter Notebook or Python file.
- Use sklearn.datasets.load_iris() only to obtain the dataset; do not use a ready-made FP-Growth or association-rule-mining function.
- Show important intermediate calculations, item ordering, FP-tree structure, conditional pattern bases, conditional trees, and final frequent itemsets.
- Include the required tables, plots, and observations.
- Use clear function definitions and meaningful variable names.
- Submit the notebook/source code along with the generated figures.

Part A: Dataset and Transaction Preparation

Dataset: Iris from sklearn.datasets. The numerical attributes must be converted into categorical items so that each Iris record can be treated as a transaction.

1. Load the Iris dataset and display the number of transactions, number of features, feature names, and class names.

2. Separate the four numerical measurements and the species/class label.

3. For each numerical feature, calculate the 1/3 and 2/3 quantiles using NumPy.

4. Convert each feature value into one of three categories: low, medium, or high. Use the first quantile as the low/medium boundary and the second quantile as the medium/high boundary.

5. Represent every record as a transaction containing five items, for example: sepal_length=low, sepal_width=high, petal_length=low, petal_width=low, and species=setosa.

6. Print at least the first 10 transactions and verify that every transaction is represented as a set of items.

Advance Data Analytics Lab - FP-Growth Algorithm

Task 1: Inspect the Transaction Database

1. Print the total number of transactions.
2. Print the complete set of unique items occurring in the transaction database.
3. Print the number of unique items.
4. For at least five selected items, calculate and display their occurrence counts.
5. Briefly explain why the converted Iris records can be treated as a transaction database for Boolean association-rule mining.

Task 2: Implement Support

For an itemset I, support is the percentage of transactions containing that itemset:

$$\text{support}(I) = \text{count}(I) / N$$

1. Write a function support(itemset, transactions) that returns the support of an itemset.
2. Test the function using at least three 1-itemsets and three 2-itemsets.
3. Print the itemset, occurrence count, and support percentage.

Task 3: Support and Confidence for an Association Rule

For a rule $A \rightarrow B$, use the following definitions:

$$\text{support}(A \rightarrow B) = P(A \cup B) \quad \text{confidence}(A \rightarrow B) = \text{support}(A \cup B) / \text{support}(A)$$

1. Write a function confidence(antecedent, consequent, transactions).
2. Choose one simple rule such as {petal_length=low} $\rightarrow$ {species=setosa} and calculate its support and confidence.
3. Show the numerical calculation manually in the notebook in addition to the Python output.
4. Explain why confidence is directional; compare $A \rightarrow B$ with $B \rightarrow A$ for one chosen pair of itemsets.

Task 4: Set Minimum Thresholds

Use the following initial thresholds for the main experiment:

$$\text{minimum support} = 0.20\ (20\%) \quad \text{minimum confidence} = 0.70\ (70\%)$$

Explain in one or two sentences what each threshold controls and why both thresholds are needed when generating strong association rules.

Part B: FP-Growth Algorithm

FP-Growth avoids generating large candidate-itemset sets. It scans the database to find frequent items, orders items by descending frequency, compresses transactions into an FP-tree, and recursively mines conditional pattern bases and conditional FP-trees.

Task 5: Find Frequent Items and Build the Header Table

1. Count the occurrence and support of every unique item.
2. Apply minimum support = 0.20 to identify frequent 1-items.
3. Remove infrequent items from each transaction.
4. Sort the remaining items in each transaction by descending global frequency; use a consistent tie-breaking rule such as alphabetical order.
5. Create a header table with columns: Item, Count, Support, and Node Link (or linked-node count).

Task 6: Implement FP-Tree Node Structure

1. Create a tree-node class containing item name, count, parent pointer, children, and node-link pointer.
2. Create a header-table structure that stores each frequent item's total count and links to all tree nodes carrying that item.

3. Implement insertion of an ordered transaction into the FP-tree, incrementing counts when a path already exists and creating nodes only when needed.

4. Print the root and the tree paths with counts in a readable indented format.

5. Explain how shared prefixes compress repeated transaction patterns.

Advance Data Analytics Lab - FP-Growth Algorithm

Task 7: Construct the FP-Tree

1. Build the FP-tree by inserting every filtered and ordered transaction.
2. Display the global frequency order used for the tree.
3. Show at least five example transactions before and after filtering/reordering.
4. Print the complete FP-tree structure with node counts.
5. Verify that the sum of counts for each item's linked nodes equals that item's frequency in the filtered transaction database.

Task 8: Implement Node-Link Traversal

The header table's node links connect all tree nodes corresponding to the same item.

1. Write a function to traverse the node-link chain for a selected item.
2. Display every node count for at least five frequent items.
3. Sum the linked-node counts and compare the sum with the item's support count from the original database.
4. Briefly explain why node links make conditional-pattern extraction efficient.

Task 9: Generate Conditional Pattern Bases

For each item, a conditional pattern base contains the prefix paths leading to that item, weighted by the count of the item's node.

1. Write a function find_prefix_paths(item, header_table) to collect prefix paths and their counts.
2. Choose at least five frequent items and print their conditional pattern bases.
3. Show each prefix path and its associated count; do not include the suffix item's own node in the prefix path.
4. Explain how the path counts represent transactions sharing that prefix.

Task 10: Build Conditional FP-Trees

1. Write a function that builds a conditional FP-tree from a conditional pattern base.
2. Count item frequencies in each conditional pattern base and remove items below minimum support.
3. Order retained items consistently and insert the weighted paths into a conditional tree.
4. Display at least three conditional FP-trees, including their suffix item and node counts.
5. Explain why a conditional tree may be smaller than the original FP-tree.

Task 11: Complete the FP-Growth Algorithm

Implement a recursive function with a structure similar to:

```
def fp_growth(tree, header_table, suffix, min_support):
# if the tree has a single path, enumerate its combinations
# otherwise process header items from least frequent to most frequent
    # append each item to the suffix pattern
    # output the pattern with its support count
    # build the item's conditional pattern base and conditional FP-tree
    # recursively mine the conditional tree
```

1. Implement the complete algorithm without using a frequent-pattern-mining library.
2. Ensure that each discovered itemset is stored once with its correct support count.
3. Handle both single-path trees and trees with branching.
4. Stop recursion when a conditional tree is empty or no conditional frequent items remain.

Task 12: Display All Frequent Itemsets

1. Run FP-Growth using minimum support = 0.20.
2. Display frequent 1-itemsets, 2-itemsets, 3-itemsets, and any higher-level itemsets discovered.
3. For each itemset, report its occurrence count and support percentage.
4. Report the total number of frequent itemsets discovered.
5. State the largest frequent itemset size found.

Advance Data Analytics Lab - FP-Growth Algorithm

Task 13: Verify Frequent-Itemset Support

1. Select at least five discovered frequent itemsets, including itemsets of size two or more.
2. Recount each selected itemset directly against the original transactions using your support function.
3. Compare the direct count with the count returned by FP-Growth.
4. Investigate and explain any mismatch; all counts should agree.
5. Explain why correct transaction filtering and weighted conditional paths are important for support correctness.

Task 14: Generate Association Rules from Frequent Itemsets

For each frequent itemset L containing at least two items, generate every non-empty proper subset S and form the rule $S \rightarrow (L - S)$. Keep the rule when its confidence satisfies the minimum-confidence threshold.

1. Write a function generate_rules(frequent_itemsets, min_confidence=0.70).
2. For every generated rule, calculate support and confidence using transaction counts.
3. Keep only rules with confidence ≥ 0.70.
4. Store the final rules in a structured form such as a list of dictionaries or tuples.

Task 15: Present the Strong Association Rules

Create a table containing at least 10 strong rules, where available, with columns:

Antecedent | Consequent | Support | Confidence

1. Sort the displayed rules by confidence in descending order and then by support in descending order.
2. Identify one rule with high support and one rule with high confidence.
3. Do not interpret a rule as causal; explain that association rules describe co-occurrence patterns.

Task 16: Manual Rule Calculation

Select one rule from your final results, for example a rule involving a petal category and a species category. Show the complete calculation using transaction counts:

$$\text{support}(A \rightarrow B) = \text{count}(A \cup B) / N \quad \text{confidence}(A \rightarrow B) = \text{count}(A \cup B) / \text{count}(A)$$

1. Write the antecedent A and consequent B.
2. Count transactions containing A.
3. Count transactions containing both A and B.
4. Calculate support and confidence manually.
5. Compare the manual result with the program output.

Task 17: Effect of Minimum Support

1. Repeat FP-Growth using minimum support values of 0.15, 0.20, and 0.25, keeping the same transaction database.
2. For each threshold, record the number of frequent 1-itemsets, 2-itemsets, 3-itemsets, and higher-level itemsets.
3. Record the total number of frequent itemsets.
4. Plot minimum support versus total number of frequent itemsets.
5. Explain how changing minimum support affects the number of patterns found and the size of conditional trees.

Advance Data Analytics Lab - FP-Growth Algorithm

Task 18: Effect of Minimum Confidence

1. Using minimum support = 0.20, repeat rule generation with minimum confidence values 0.50, 0.70, and 0.90.
2. Record the number of strong rules produced at each confidence threshold.
3. Plot minimum confidence versus number of strong rules.
4. Explain why increasing the minimum-confidence threshold usually reduces the number of accepted rules.

Task 19: Required Visualizations

1. Create a bar chart showing the number of frequent itemsets at each itemset size k.
2. Create a plot showing minimum support versus the total number of frequent itemsets for the three support thresholds.
3. Create a plot showing minimum confidence versus the number of strong rules for the three confidence thresholds.
4. Create a support-versus-confidence scatter plot for the final strong rules using minimum support = 0.20 and minimum confidence = 0.70.
5. Include a visual representation of the FP-tree or a clearly labelled tree diagram for the main experiment.

All plots must have labelled axes, a suitable title, and readable tick labels. Include a short observation below each plot.

Task 20: Compare FP-Growth with Apriori

1. Explain the main steps of FP-Growth: frequent-item scan, frequency ordering, FP-tree construction, conditional pattern base/tree generation, and recursive mining.
2. Compare FP-Growth with Apriori in terms of candidate generation, database scans, memory use, and implementation complexity.
3. Explain why FP-Growth generally avoids the candidate explosion that can occur in Apriori.
4. State a situation where FP-Growth may still be challenging, such as a tree that cannot compress the data well or a tree that requires substantial memory.

Task 21: Algorithmic Complexity and Practical Observation

1. Record the number of FP-tree nodes in the main tree and in at least three conditional trees.
2. Record the number of frequent itemsets mined and the maximum itemset size.
3. Comment on how shared prefixes affect tree size and how conditional mining reduces the search space.
4. Give one practical advantage and one limitation of FP-Growth.
5. Explain why runtime and memory use depend on transaction length, item frequency distribution, and pattern density.

Task 22: Final Results Table

Prepare a compact final summary table with the following information:

Experiment	Setting	Result to Report
Main FP-Growth run	min support = 0.20	Total frequent itemsets; largest k; FP-tree node count
Rule generation	min confidence = 0.70	Number of strong rules
Support experiment	0.15 / 0.20 / 0.25	Frequent itemsets at each level
Confidence experiment	0.50 / 0.70 / 0.90	Number of strong rules
Conditional mining	At least 3 suffix items	Conditional pattern bases/trees examined

Advance Data Analytics Lab - FP-Growth Algorithm

Task 23: Observations and Conclusion

1. Write 4–6 observations about the frequent itemsets discovered in the Iris transaction database.

2. Write 2–3 observations about support and confidence.

3. Explain how FP-tree compression and conditional pattern bases help mine frequent patterns without generating candidate itemsets level by level.

4. Mention how changing minimum support and minimum confidence affects the final results.

5. Conclude with a brief statement about where frequent-pattern mining can be useful, such as market-basket analysis, shelf management, inventory control, document analysis, or other applications discussed in the lecture.

Submission Checklist

- Single Jupyter Notebook or Python source file.
- Dataset loading and transaction-conversion code.
- FP-Growth implementation from scratch.
- Support and confidence calculations.
- Frequency ordering, header table, FP-tree and node-link outputs.
- Conditional pattern bases and conditional FP-trees.
- Frequent-itemset and strong-rule tables.
- Required plots and written observations.
- Final results summary and conclusion.

Important Note

Do not use mlxtend.frequent_patterns.fpgrowth, mlxtend.frequent_patterns.association_rules, pyfpgrowth, or another ready-made frequent-pattern-mining implementation for the main solution. The purpose of this assignment is to demonstrate FP-Growth: frequency ordering, FP-tree construction, node links, conditional pattern bases and trees, recursive pattern mining, support counting, and generation of strong association rules.

Reference within the lecture material: Support and Confidence; Frequent Pattern Growth (FP-Growth); FP-tree construction; conditional pattern bases and conditional FP-trees; and comparison of Apriori with FP-Growth.